Business Formation Analysis Project

This notebook is a starting point for exploring business formation data in the USA. The goal is to create a RandomForest model that predicts current week state level business formation. Based on state data (excluding current week BA_NSA - see the feature name legend below) AND national formation data including up to current week. Additionally some exploratory statistics of the data are made including graphs and the secondary aim is to identify if some of the features are more useful to predict the state level current week applications counts.

More precisely: the random forest model will classify current week state level BA_NSA as an increase, decrease or constant compared to the previous week. Without seeing the current week BA_NSA value.

Questions

- Can current week state level business formation be predicted accurately? (INCREASE vs DECREASE vs CONSTANT)
- Which features have the highest explanatory power for current week state level application.

Data and analysis

The data source is Business Formation Statistics (BFS) from the U.S. Census Bureau at: www.census.gov

Two .csv datasets are used:

- bfs_us_apps_weekly_nsa.csv
- bfs_state_apps_weekly_nsa.csv

```
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import scipy
import sklearn
from scipy import stats
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix
from sklearn.utils.validation import check_array
from IPython.display import display

#pd.set_option(&amp;quot;display.max_columns&amp;quot;, 30)
#pd.set_option(&amp;quot;display.width&amp;quot;, 140)

TIME_KEYS = [&amp;quot;Year&amp;quot;, &amp;quot;Week&amp;quot;]

us_raw = pd.read_csv(&amp;quot;bfs_us_apps_weekly_nsa.csv&amp;quot;)
state_raw = pd.read_csv(&amp;quot;bfs_state_apps_weekly_nsa.csv&amp;quot;)

us_raw.head()
```

state_raw.head()

Reshape state data and merge national features

```
# Validate and normalize the time keys before reshaping or joining.
required_state = set(TIME_KEYS + [&quot;State&quot;])
required_us = set(TIME_KEYS)
if not required_state.issubset(state_raw.columns):
    raise ValueError(f&quot;State file is missing required columns: {sorted(required_state - set(state_raw.columns))}&quot;)
if not required_us.issubset(us_raw.columns):
    raise ValueError(f&quot;U.S. file is missing required columns: {sorted(required_us - set(us_raw.columns))}&quot;)

state_data = state_raw.copy()
us_data = us_raw.copy()
for frame in (state_data, us_data):
    for key in TIME_KEYS:
        frame[key] = pd.to_numeric(frame[key], errors=&quot;raise&quot;).astype(&quot;int64&quot;)

state_data[&quot;State&quot;] = state_data[&quot;State&quot;].astype(&quot;string&quot;).str.strip()
if state_data[&quot;State&quot;].isna().any() or state_data[&quot;State&quot;].eq(&quot;&quot;).any():
    raise ValueError(&quot;State file contains a missing or blank State value.&quot;)

state_features = [column for column in state_data.columns if column not in TIME_KEYS + [&quot;State&quot;]]
us_features = [column for column in us_data.columns if column not in TIME_KEYS]
if not state_features or not us_features:
    raise ValueError(&quot;Expected at least one non-time feature in each source file.&quot;)

for column in state_features:
    state_data[column] = pd.to_numeric(state_data[column], errors=&quot;raise&quot;)
for column in us_features:
    us_data[column] = pd.to_numeric(us_data[column], errors=&quot;raise&quot;)

state_key = TIME_KEYS + [&quot;State&quot;]
if state_data.duplicated(state_key).any():
    examples = state_data.loc[state_data.duplicated(state_key, keep=False), state_key].head()
    raise ValueError(f&quot;State data has duplicate state-week records. Examples:\n{examples}&quot;)
if us_data.duplicated(TIME_KEYS).any():
    examples = us_data.loc[us_data.duplicated(TIME_KEYS, keep=False), TIME_KEYS].head()
    raise ValueError(f&quot;U.S. data has duplicate week records. Examples:\n{examples}&quot;)

# pivot produces (feature, State) column pairs; flip that order for State_feature names.
state_wide = state_data.pivot(index=TIME_KEYS, columns=&quot;State&quot;, values=state_features)
state_wide.columns = [f&quot;{state}_{feature}&quot; for feature, state in state_wide.columns]
state_wide = state_wide.reset_index()

us_prefixed = us_data.rename(columns={column: f&quot;USA_{column}&quot; for column in us_features})

# Outer join preserves every observed week from either source; unmatched weeks remain visible as missing values.
combined = state_wide.merge(
    us_prefixed,
    on=TIME_KEYS,
    how=&quot;outer&quot;,
    validate=&quot;one_to_one&quot;,
    indicator=&quot;_source_match&quot;,
).sort_values(TIME_KEYS).reset_index(drop=True)

source_match_counts = combined[&quot;_source_match&quot;].value_counts().reindex(
    [&quot;both&quot;, &quot;left_only&quot;, &quot;right_only&quot;], fill_value=0
)
combined = combined.drop(columns=&quot;_source_match&quot;)

#print(f&quot;State-wide data: {state_wide.shape[0]:,} weeks x {state_wide.shape[1]} columns&quot;)
#print(f&quot;Merged data: {combined.shape[0]:,} weeks x {combined.shape[1]} columns&quot;)
#print(&quot;Week coverage (both / state only / U.S. only):&quot;, source_match_counts.to_dict())
#print(&quot;States:&quot;, state_data[&quot;State&quot;].nunique())
```

```
#first_week = combined.iloc[0]
#last_week = combined.iloc[-1]
#print(f&amp;quot;Time-key range: {int(first_week[&amp;#x27;Year&amp;#x27;])} week {int(first_week[&amp;#x27;Week&amp;#x27;])} &amp;quot;
#      f&amp;quot;to {int(last_week[&amp;#x27;Year&amp;#x27;])} week {int(last_week[&amp;#x27;Week&amp;#x27;])}&amp;quot;)
#display(combined.head())
```

Weekly application trends

The plot showcases BA_NSA as an initial headline measure. Counts are not seasonally adjusted, so recurring within-year patterns should be kept in mind.

```
fig, axes = plt.subplots(2, 1, figsize=(14, 9), constrained_layout=True)

if &amp;quot;USA_BA_NSA&amp;quot; in combined.columns:
    national = combined.dropna(subset=[&amp;quot;USA_BA_NSA&amp;quot;]).copy()
    national[&amp;quot;Time&amp;quot;] = national[&amp;quot;Year&amp;quot;].astype(str) + &amp;quot;-W&amp;quot; + national[&amp;quot;Week&amp;quot;].astype(str).str.zfill(2)
    axes[0].plot(national[&amp;quot;Time&amp;quot;], national[&amp;quot;USA_BA_NSA&amp;quot;], color=&amp;quot;navy&amp;quot;, linewidth=1.2)
    axes[0].set_title(&amp;quot;United States: weekly business applications (BA_NSA)&amp;quot;)
    axes[0].set_ylabel(&amp;quot;Applications&amp;quot;)
    axes[0].grid(alpha=0.25)
    tick_step = max(1, len(national) // 12)
    axes[0].set_xticks(range(0, len(national), tick_step))
    axes[0].set_xticklabels(national[&amp;quot;Time&amp;quot;].iloc[::tick_step], rotation=45, ha=&amp;quot;right&amp;quot;)

axes[1].set_visible(False)

plt.show()
```

Combined data quality and distribution analysis

Check for Nans and conduct normality testing.

```
missing_counts = combined.isna().sum().sort_values(ascending=False)
missing_summary = missing_counts.rename(&amp;quot;missing_count&amp;quot;).to_frame()
missing_summary[&amp;quot;missing_fraction&amp;quot;] = missing_summary[&amp;quot;missing_count&amp;quot;] / len(combined)
print(&amp;quot;Missing values by column (all columns):&amp;quot;)
display(missing_summary)

time_values = combined[[&amp;quot;Year&amp;quot;, &amp;quot;Week&amp;quot;]].to_numpy(dtype=np.float64)
time_keys_present = np.isfinite(time_values).all()
if not time_keys_present:
    chronology_ok = False
    duplicate_time_key_count = np.nan
    out_of_order_pair_count = np.nan
else:
    years = time_values[:, 0]
    weeks = time_values[:, 1]
    duplicate_time_key_count = int(combined.duplicated([&amp;quot;Year&amp;quot;, &amp;quot;Week&amp;quot;]).sum())
    out_of_order_pairs = (years[1:] &amp;lt; years[:-1]) | (
        (years[1:] == years[:-1]) &amp;amp; (weeks[1:] &amp;lt; weeks[:-1])
    )
    out_of_order_pair_count = int(np.count_nonzero(out_of_order_pairs))
    chronology_ok = duplicate_time_key_count == 0 and out_of_order_pair_count == 0

feature_columns = [
    column
    for column in combined.select_dtypes(include=[np.number]).columns
    if column not in [&amp;quot;Year&amp;quot;, &amp;quot;Week&amp;quot;]
]
if not feature_columns:
    raise ValueError(&amp;quot;No numeric feature columns were found in combined.&amp;quot;)
```

```
feature_matrix = combined[feature_columns].to_numpy(dtype=np.float64)
statistics_rows = []
for column_index, column in enumerate(feature_columns):
    values = feature_matrix[:, column_index]
    valid_values = values[np.isfinite(values)]
    if valid_values.size:
        # Validate finite observations with scikit-learn before numerical calculations.
        valid_values = check_array(valid_values.reshape(-1, 1), dtype=np.float64).ravel()
        mean = float(np.mean(valid_values))
        st_dev = float(np.std(valid_values, ddof=1)) if valid_values.size &gt; 1 else np.nan
    else:
        mean = np.nan
        st_dev = np.nan

    if valid_values.size &gt;= 3 and st_dev &gt; 0:
        jb_result = stats.jarque_bera(valid_values)
        jb_statistic = float(jb_result.statistic)
        jb_p_value = float(jb_result.pvalue)
        normality_result = &quot;Reject normality&quot; if jb_p_value &lt; 0.05 else &quot;Fail to reject normality&quot;
    else:
        jb_statistic = np.nan
        jb_p_value = np.nan
        normality_result = &quot;Not testable (fewer than 3 finite values or zero variance)&quot;

    statistics_rows.append(
        {
            &quot;feature&quot;: column,
            &quot;n_finite&quot;: int(valid_values.size),
            &quot;mean&quot;: mean,
            &quot;st_dev&quot;: st_dev,
            &quot;jarque_bera_statistic&quot;: jb_statistic,
            &quot;jarque_bera_p_value&quot;: jb_p_value,
            &quot;normality_at_0.05&quot;: normality_result,
        }
    )

feature_statistics = pd.DataFrame(statistics_rows).set_index(&quot;feature&quot;)
feature_statistics

plot_features = [
    column for column in (&quot;USA_BA_NSA&quot;, &quot;CA_BA_NSA&quot;)
    if column in feature_columns and np.isfinite(feature_matrix[:, feature_columns.index(column)]).sum() &gt;= 3
]
if not plot_features:
    plot_features = [
        column for column in feature_columns
        if np.isfinite(feature_matrix[:, feature_columns.index(column)]).sum() &gt;= 3
    ][:2]

if not plot_features:
    print(&quot;No feature has enough finite values to create normality plots.&quot;)
else:
    figure, axes = plt.subplots(len(plot_features), 2, figsize=(12, 4 * len(plot_features)), squeeze=False)
    for row, column in enumerate(plot_features):
        values = feature_matrix[:, feature_columns.index(column)]
        values = values[np.isfinite(values)]
        mean = np.mean(values)
        st_dev = np.std(values, ddof=1)

        axes[row, 0].hist(
            values,
            bins=min(40, max(10, int(np.sqrt(values.size)))),
            density=True,
```

```
            alpha=0.7,
            color=&quot;steelblue&quot;,
            edgecolor=&quot;white&quot;,
        )
        if st_dev &gt; 0:
            x_grid = np.linspace(np.min(values), np.max(values), 300)
            axes[row, 0].plot(x_grid, stats.norm.pdf(x_grid, loc=mean, scale=st_dev), color=&quot;darkred&quot;, linewidth=2)
        axes[row, 0].set_title(f&quot;{column}: histogram and fitted normal density&quot;)
        axes[row, 0].set_xlabel(&quot;Observed value&quot;)
        axes[row, 0].set_ylabel(&quot;Density&quot;)
        axes[row, 0].grid(alpha=0.2)

        stats.probplot(values, dist=&quot;norm&quot;, plot=axes[row, 1])
        axes[row, 1].set_title(f&quot;{column}: normal Q-Q plot&quot;)
        axes[row, 1].grid(alpha=0.2)

    figure.tight_layout()
    plt.show()
```

Random forest classification data preparation

The target is the direction of change in each state's weekly BA_NSA applications versus its preceding observation: DECREASED, CONSTANT, or INCREASED. For each target week, predictors include the preceding 52 state-week observations for that state's available features, contemporaneous USA_ features, and a one-hot state identifier. The target week's state values are never predictors. Samples are split chronologically by target year: 2006–2023 for training and 2024–2026 for testing. Missing predictor values are imputed using training-set medians only.

```
LAG_WEEKS = 52
CLASS_NAMES = np.array([&quot;DECREASED&quot;, &quot;CONSTANT&quot;, &quot;INCREASED&quot;])
CLASS_TO_CODE = {name: code for code, name in enumerate(CLASS_NAMES)}

if combined.duplicated([&quot;Year&quot;, &quot;Week&quot;]).any():
    raise ValueError(&quot;combined must have one row per Year/Week before creating model samples.&quot;)
time_values = combined[[&quot;Year&quot;, &quot;Week&quot;]].to_numpy(dtype=np.int64)
if np.any((time_values[1:, 0] &lt; time_values[:-1, 0]) | (
    (time_values[1:, 0] == time_values[:-1, 0]) &amp;amp; (time_values[1:, 1] &lt; time_values[:-1, 1])
)):
    raise ValueError(&quot;combined must be in chronological Year/Week order before creating lag features.&quot;)

state_names = sorted(state_data[&quot;State&quot;].dropna().unique().tolist())
state_feature_names = list(state_features)
national_feature_names = [column for column in combined.columns if column.startswith(&quot;USA_&quot;)]
missing_model_columns = [
    f&quot;{state}_{feature}&quot;
    for state in state_names
    for feature in state_feature_names
    if f&quot;{state}_{feature}&quot; not in combined.columns
] + [column for column in national_feature_names if column not in combined.columns]
if missing_model_columns or &quot;BA_NSA&quot; not in state_feature_names:
    raise ValueError(f&quot;Required model features are unavailable: {missing_model_columns[:10]}&quot;)
if not national_feature_names:
    raise ValueError(&quot;No USA-prefixed national features are available.&quot;)
if len(combined) &lt;= LAG_WEEKS:
    raise ValueError(f&quot;At least {LAG_WEEKS + 1} weekly observations are required.&quot;)

state_input_columns = [f&quot;{state}_{feature}&quot; for feature in state_feature_names for state in state_names]
state_input_matrix = combined[state_input_columns].to_numpy(dtype=np.float64).reshape(
    len(combined), len(state_feature_names), len(state_names)
)
national_input_matrix = combined[national_feature_names].to_numpy(dtype=np.float64)
application_feature_index = state_feature_names.index(&quot;BA_NSA&quot;)
```

```
state_codes = {state: index for index, state in enumerate(state_names)}

lag_feature_names = [
    f&amp;quot;lag_{lag:02d}_{feature}&amp;quot;
    for lag in range(LAG_WEEKS, 0, -1)
    for feature in state_feature_names
]
model_feature_names = lag_feature_names + national_feature_names + [f&amp;quot;state_{state}&amp;quot; for state in state_names]
sample_features = []
sample_labels = []
sample_years = []
sample_weeks = []
sample_states = []

for state in state_names:
    state_index = state_codes[state]
    state_applications = state_input_matrix[:, application_feature_index, state_index]
    state_indicator = np.zeros(len(state_names), dtype=np.float64)
    state_indicator[state_index] = 1.0

    for target_index in range(LAG_WEEKS, len(combined)):
        previous_applications = state_applications[target_index - 1]
        current_applications = state_applications[target_index]
        if not np.isfinite(previous_applications) or not np.isfinite(current_applications):
            continue

        change = np.sign(current_applications - previous_applications)
        class_code = int(change + 1)
        historical_features = state_input_matrix[target_index - LAG_WEEKS:target_index, :, state_index].reshape(-1)
        current_national_features = national_input_matrix[target_index]
        sample_features.append(np.concatenate((historical_features, current_national_features, state_indicator)))
        sample_labels.append(class_code)
        sample_years.append(int(time_values[target_index, 0]))
        sample_weeks.append(int(time_values[target_index, 1]))
        sample_states.append(state)

X = np.asarray(sample_features, dtype=np.float64)
y = np.asarray(sample_labels, dtype=np.int8)
sample_years = np.asarray(sample_years, dtype=np.int64)
sample_weeks = np.asarray(sample_weeks, dtype=np.int64)
sample_states = np.asarray(sample_states, dtype=str)

train_mask = sample_years &amp;lt;= 2023
test_mask = (sample_years &amp;gt;= 2024) &amp;amp; (sample_years &amp;lt;= 2026)
if not train_mask.any() or not test_mask.any():
    raise ValueError(&amp;quot;The dataset must contain usable training observations through 2023 and test observations in 2024–2026.&amp;quot;)
if np.any(~np.isfinite(sample_years)):
    raise ValueError(&amp;quot;Model sample years must be finite.&amp;quot;)

X_train_raw = X[train_mask]
X_test_raw = X[test_mask]
y_train = y[train_mask]
y_test = y[test_mask]
train_feature_medians = np.nanmedian(
    np.where(np.isfinite(X_train_raw), X_train_raw, np.nan), axis=0
)
train_feature_medians = np.where(np.isfinite(train_feature_medians), train_feature_medians, 0.0)
X_train = np.where(np.isfinite(X_train_raw), X_train_raw, train_feature_medians)
X_test = np.where(np.isfinite(X_test_raw), X_test_raw, train_feature_medians)
X_train = check_array(X_train, dtype=np.float64)
X_test = check_array(X_test, dtype=np.float64)

print(f&amp;quot;Samples: {len(y):,}; features: {len(model_feature_names):,}&amp;quot;)
```

```
print(f&amp;quot;Training samples (through 2023): {len(y_train):,}&amp;quot;)
print(f&amp;quot;Testing samples (2024–2026): {len(y_test):,}&amp;quot;)
print(&amp;quot;Training class counts:&amp;quot;, dict(zip(CLASS_NAMES, np.bincount(y_train, minlength=len(CLASS_NAMES)))))
print(&amp;quot;Testing class counts:&amp;quot;, dict(zip(CLASS_NAMES, np.bincount(y_test, minlength=len(CLASS_NAMES)))))
```

Training the random forest

Adjust MAX_TREE_DEPTH to tune the maximum depth. The random forest is fitted only on the training period (through 2023); the same model object is retained for test-set prediction.

```
MAX_TREE_DEPTH = 7

model = RandomForestClassifier(
    n_estimators=100,
    max_depth=MAX_TREE_DEPTH,
    random_state=42,
    n_jobs=-1,
)
model.fit(X_train, y_train)

print(f&amp;quot;Fitted {model.__class__.__name__} on {len(y_train):,} samples.&amp;quot;)
print(f&amp;quot;Maximum tree depth: {MAX_TREE_DEPTH}&amp;quot;)
```

Training-set in-sample evaluation

These predictions are made on the same training observations used to fit the model, so they are an in-sample diagnostic and can be optimistic. The confusion matrix uses actual classes as rows and predicted classes as columns. TP/FP/TN/FN are also reported one-vs-rest for each direction class.

```
train_predictions = model.predict(X_train)
train_confusion_matrix = confusion_matrix(y_train, train_predictions, labels=np.arange(len(CLASS_NAMES)))
train_accuracy = np.trace(train_confusion_matrix) / train_confusion_matrix.sum()
train_class_results = []
for class_index, class_name in enumerate(CLASS_NAMES):
    true_positive = int(train_confusion_matrix[class_index, class_index])
    false_positive = int(train_confusion_matrix[:, class_index].sum() - true_positive)
    false_negative = int(train_confusion_matrix[class_index, :].sum() - true_positive)
    true_negative = int(train_confusion_matrix.sum() - true_positive - false_positive - false_negative)
    class_recall = true_positive / (true_positive + false_negative) if true_positive + false_negative else np.nan
    train_class_results.append(
        {&amp;quot;class&amp;quot;: class_name, &amp;quot;TP&amp;quot;: true_positive, &amp;quot;FP&amp;quot;: false_positive, &amp;quot;TN&amp;quot;: true_negative, &amp;quot;FN&amp;quot;: false_negative,
&amp;quot;recall&amp;quot;: class_recall}
    )

print(f&amp;quot;Training in-sample accuracy: {train_accuracy:.4f}&amp;quot;)
print(f&amp;quot;Training macro recall: {np.nanmean([row[&amp;#x27;recall&amp;#x27;] for row in train_class_results]):.4f}&amp;quot;)
display(pd.DataFrame(train_confusion_matrix, index=CLASS_NAMES, columns=CLASS_NAMES).rename_axis(&amp;quot;Actual \\ Predicted&amp;quot;))
display(pd.DataFrame(train_class_results).set_index(&amp;quot;class&amp;quot;))
```

Testing on 2024–2026

Predict the held-out 2024–2026 observations with the model fitted on data through 2023. No refitting occurs in this section. Accuracy is overall multiclass accuracy; recall is reported per class and as macro recall across the three direction classes.

```
test_predictions = model.predict(X_test)
test_confusion_matrix = confusion_matrix(y_test, test_predictions, labels=np.arange(len(CLASS_NAMES)))
test_accuracy = np.trace(test_confusion_matrix) / test_confusion_matrix.sum()
test_class_results = []
```

```
for class_index, class_name in enumerate(CLASS_NAMES):
    true_positive = int(test_confusion_matrix[class_index, class_index])
    false_positive = int(test_confusion_matrix[:, class_index].sum() - true_positive)
    false_negative = int(test_confusion_matrix[class_index, :].sum() - true_positive)
    true_negative = int(test_confusion_matrix.sum() - true_positive - false_positive - false_negative)
    class_recall = true_positive / (true_positive + false_negative) if true_positive + false_negative else np.nan
    test_class_results.append(
        {&quot;class&quot;: class_name, &quot;TP&quot;: true_positive, &quot;FP&quot;: false_positive, &quot;TN&quot;: true_negative, &quot;FN&quot;: false_negative,
&quot;recall&quot;: class_recall}
    )

print(f&quot;Test-set accuracy: {test_accuracy:.4f}&quot;)
print(f&quot;Test macro recall: {np.nanmean([row[&#x27;recall&#x27;] for row in test_class_results]):.4f}&quot;)
display(pd.DataFrame(test_confusion_matrix, index=CLASS_NAMES, columns=CLASS_NAMES).rename_axis(&quot;Actual \\ Predicted&quot;))
display(pd.DataFrame(test_class_results).set_index(&quot;class&quot;))
```

Feature strength

Random-forest MDI importances provides a first view of which lagged state features, current-week national features, and state identifiers are best for tree splits.

```
feature_importance = pd.DataFrame(
    {&quot;feature&quot;: model_feature_names, &quot;importance&quot;: model.feature_importances_}
).sort_values(&quot;importance&quot;, ascending=False, ignore_index=True)
display(feature_importance.head(30).set_index(&quot;feature&quot;))

feature_importance[&quot;feature_group&quot;] = np.select(
    [
        feature_importance[&quot;feature&quot;].str.startswith(&quot;USA_&quot;),
        feature_importance[&quot;feature&quot;].str.startswith(&quot;state_&quot;),
    ],
    [&quot;Current-week national features&quot;, &quot;State identity&quot;],
    default=&quot;Lagged state features&quot;,
)
group_importance = feature_importance.groupby(&quot;feature_group&quot;, as_index=True)[&quot;importance&quot;].sum().sort_values(ascending=False)
print(&quot;Total importance by feature group:&quot;)
display(group_importance.to_frame())

top_features = feature_importance.head(20).sort_values(&quot;importance&quot;)
plt.figure(figsize=(10, 8))
plt.barh(top_features[&quot;feature&quot;], top_features[&quot;importance&quot;], color=&quot;steelblue&quot;)
plt.title(&quot;Top 20 random-forest features&quot;)
plt.xlabel(&quot;Mean decrease in impurity importance&quot;)
plt.ylabel(&quot;Feature&quot;)
plt.grid(axis=&quot;x&quot;, alpha=0.25)
plt.tight_layout()
plt.show()
```